

31.yolov8模型部署(优化加速版)

1. YOLOv8原版测试

注：本节课用的模型，数据集，代码等较多，全部会在网盘仓库中包含，学者可以跟着文档从头来一遍，也可以拿转换好的模型直接来进行测试和实验。

1.1 获取onnx模型

这个部分与上节课yolov26的一样。在这里我也就不过多的啰嗦了，我们只要拿到yolov8s/n.onnx的模型就可以了。

网盘中提供一个yolov8的文件，里面有onnx模型大家直接拿来用就行。

1.2 转换为rknn模型

执行 python convert_rknn.py 转换得到rknn 模型。

这里可以分别生成量化前和量化后的模型，方便我们后期进行对比。

这里有IOU的具体使用，只要yolov26没有，其他版本基本都有。

代码块

```
1  import os
2  import urllib
3  import traceback
4  import time
5  import sys
6  import numpy as np
7  import cv2
8  from rknn.api import RKNN
9  from math import exp
10
11  ONNX_MODEL = './yolov8s.onnx'
12  RKNN_MODEL = './yolov8s.int.rknn'
13  DATASET = './datasets.txt'
14
15  QUANTIZE_ON = True
16
17  CLASSES = ['person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus',
18             'train', 'truck', 'boat', 'traffic light',
19             'fire hydrant', 'stop sign', 'parking meter', 'bench', 'bird',
20             'cat', 'dog', 'horse', 'sheep', 'cow',
```

```
19         'elephant', 'bear', 'zebra', 'giraffe', 'backpack', 'umbrella',
    'handbag', 'tie', 'suitcase', 'frisbee',
20         'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat',
    'baseball glove', 'skateboard', 'surfboard',
21         'tennis racket', 'bottle', 'wine glass', 'cup', 'fork', 'knife',
    'spoon', 'bowl', 'banana', 'apple',
22         'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'pizza',
    'donut', 'cake', 'chair', 'couch',
23         'potted plant', 'bed', 'dining table', 'toilet', 'tv', 'laptop',
    'mouse', 'remote', 'keyboard', 'cell phone',
24         'microwave', 'oven', 'toaster', 'sink', 'refrigerator', 'book',
    'clock', 'vase', 'scissors', 'teddy bear',
25         'hair drier', 'toothbrush']
26
27 # CLASSES = ['pedestrians', 'riders', 'partially-visible-person', 'ignore-
    regions', 'crowd']
28
29 meshgrid = []
30
31 class_num = len(CLASSES)
32 headNum = 3
33 strides = [8, 16, 32]
34 mapSize = [[80, 80], [40, 40], [20, 20]] # 网格大小
35 nmsThresh = 0.5
36 objectThresh = 0.2
37
38 input_imgH = 640
39 input_imgW = 640
40
41 class DetectBox:
42     def __init__(self, classId, score, xmin, ymin, xmax, ymax, head):
43         self.classId = classId
44         self.score = score
45         self.xmin = xmin
46         self.ymin = ymin
47         self.xmax = xmax
48         self.ymax = ymax
49         self.head = head
50
51     def GenerateMeshgrid():
52         for index in range(headNum):
53             for i in range(mapSize[index][0]):
54                 for j in range(mapSize[index][1]):
55                     meshgrid.append(j + 0.5)
56                     meshgrid.append(i + 0.5)
57
58     def IOU(xmin1, ymin1, xmax1, ymax1, xmin2, ymin2, xmax2, ymax2):
```

```

59     xmin = max(xmin1, xmin2)
60     ymin = max(ymin1, ymin2)
61     xmax = min(xmax1, xmax2)
62     ymax = min(ymax1, ymax2)
63
64     innerWidth = xmax - xmin
65     innerHeight = ymax - ymin
66
67     innerWidth = innerWidth if innerWidth > 0 else 0
68     innerHeight = innerHeight if innerHeight > 0 else 0
69
70     innerArea = innerWidth * innerHeight
71
72     area1 = (xmax1 - xmin1) * (ymax1 - ymin1)
73     area2 = (xmax2 - xmin2) * (ymax2 - ymin2)
74
75     total = area1 + area2 - innerArea
76
77     return innerArea / total
78
79 def NMS(detectResult):
80     predBoxes = []
81
82     sort_detectboxes = sorted(detectResult, key=lambda x: x.score, reverse=True)
83
84     for i in range(len(sort_detectboxes)):
85         xmin1 = sort_detectboxes[i].xmin
86         ymin1 = sort_detectboxes[i].ymin
87         xmax1 = sort_detectboxes[i].xmax
88         ymax1 = sort_detectboxes[i].ymax
89         classId = sort_detectboxes[i].classId
90
91         if sort_detectboxes[i].classId != -1:
92             predBoxes.append(sort_detectboxes[i])
93             for j in range(i + 1, len(sort_detectboxes), 1):
94                 if classId == sort_detectboxes[j].classId:
95                     xmin2 = sort_detectboxes[j].xmin
96                     ymin2 = sort_detectboxes[j].ymin
97                     xmax2 = sort_detectboxes[j].xmax
98                     ymax2 = sort_detectboxes[j].ymax
99                     iou = IOU(xmin1, ymin1, xmax1, ymax1, xmin2, ymin2, xmax2,
100                             ymax2)
101
102                     if iou > nmsThresh:
103                         sort_detectboxes[j].classId = -1
104
105     return predBoxes
106
107 def sigmoid(x):

```

```

105     return 1 / (1 + exp(-x))
106
107 def postprocess(out, img_h, img_w):
108     # print(meshgrid)
109     print('postprocess ... ')
110
111     detectResult = []
112     output = []
113     for i in range(len(out)):
114         output.append(out[i].reshape((-1)))
115
116     scale_h = img_h / input_imgH
117     scale_w = img_w / input_imgW
118
119     gridIndex = -2
120
121     # 遍历每个head
122     for index in range(headNum):
123         reg = output[index * 2 + 0] # 分别获取第0、2、4个head的reg, 即回归框
124         cls = output[index * 2 + 1] # 分别获取第1、3、5个head的cls, 即类别置信度
125         print('reg average, min, max:', np.average(reg), np.min(reg),
np.max(reg))
126         print('cls average, min, max:', np.average(cls), np.min(cls),
np.max(cls))
127         # 遍历每个网格的h
128         for h in range(mapSize[index][0]):
129             # 遍历每个网格的w
130             for w in range(mapSize[index][1]):
131                 gridIndex += 2
132                 # 遍历每个类别
133                 for cl in range(class_num):
134                     # 计算每个类别的置信度
135                     cls_val = sigmoid(cls[cl * mapSize[index][0] *
mapSize[index][1] + h * mapSize[index][1] + w])
136                     # 如果置信度大于阈值, 则进行后续处理
137                     if cls_val > objectThresh:
138                         # 计算回归框的坐标
139                         # 计算x1:
140                         x1 = (meshgrid[gridIndex + 0] - reg[0 * mapSize[index]
[0] * mapSize[index][1] + h * mapSize[index][1] + w]) * strides[index]
141                         y1 = (meshgrid[gridIndex + 1] - reg[1 * mapSize[index]
[0] * mapSize[index][1] + h * mapSize[index][1] + w]) * strides[index]
142                         x2 = (meshgrid[gridIndex + 0] + reg[2 * mapSize[index]
[0] * mapSize[index][1] + h * mapSize[index][1] + w]) * strides[index]
143                         y2 = (meshgrid[gridIndex + 1] + reg[3 * mapSize[index]
[0] * mapSize[index][1] + h * mapSize[index][1] + w]) * strides[index]
144

```

```

145         xmin = x1 * scale_w
146         ymin = y1 * scale_h
147         xmax = x2 * scale_w
148         ymax = y2 * scale_h
149
150         xmin = xmin if xmin > 0 else 0
151         ymin = ymin if ymin > 0 else 0
152         xmax = xmax if xmax < img_w else img_w
153         ymax = ymax if ymax < img_h else img_h
154
155         box = DetectBox(cl, cls_val, xmin, ymin, xmax, ymax,
index)
156         detectResult.append(box)
157     # NMS
158     print('detectResult:', len(detectResult))
159     predBox = NMS(detectResult)
160
161     return predBox
162
163 def export_rknn_inference(img):
164     # Create RKNN object
165     rknn = RKNN(verbose=False)
166
167     # pre-process config
168     print('--> Config model')
169     rknn.config(mean_values=[[0, 0, 0]], std_values=[[255, 255, 255]],
quantized_algorithm='normal', quantized_method='channel', optimization_level =
2, target_platform='rk3588')
170     print('done')
171
172     # Load ONNX model
173     print('--> Loading model')
174     ret = rknn.load_onnx(model=ONNX_MODEL, outputs=
["reg1", "cls1", "reg2", "cls2", "reg3", "cls3"])
175     #ret = rknn.load_onnx(model=ONNX_MODEL, outputs=['output0', '276', '311',
'300', '332', '321'])
176     if ret != 0:
177         print('Load model failed!')
178         exit(ret)
179     print('done')
180
181     # Build model
182     print('--> Building model')
183     ret = rknn.build(do_quantization=QUANTIZE_ON, dataset=DATASET,
rknn_batch_size=1)
184     if ret != 0:
185         print('Build model failed!')

```

```
186         exit(ret)
187     print('done')
188
189     # Export RKNN model
190     print('--> Export rknn model')
191     ret = rknn.export_rknn(RKNN_MODEL)
192     if ret != 0:
193         print('Export rknn model failed!')
194         exit(ret)
195     print('done')
196
197     # Init runtime environment
198     print('--> Init runtime environment')
199     ret = rknn.init_runtime()
200     # ret = rknn.init_runtime(target='rk3566')
201     if ret != 0:
202         print('Init runtime environment failed!')
203         exit(ret)
204     print('done')
205
206     # Inference
207     print('--> Running model')
208     outputs = rknn.inference(inputs=[img])
209     rknn.release()
210     print('done')
211
212     return outputs
213
214 if __name__ == '__main__':
215     print('This is main ...')
216     GenerateMeshgrid()
217
218     img_path = './zidane.jpg'
219     orig_img = cv2.imread(img_path)
220     img_h, img_w = orig_img.shape[:2]
221
222     origimg = cv2.resize(orig_img, (input_imgW, input_imgH),
223 interpolation=cv2.INTER_LINEAR)
224     origimg = cv2.cvtColor(origimg, cv2.COLOR_BGR2RGB)
225
226     img = np.expand_dims(origimg, 0)
227
228     outputs = export_rknn_inference(img)
229
230     out = []
231     for i in range(len(outputs)):
232         out.append(outputs[i])
```

```

232
233     predbox = postprocess(out, img_h, img_w)
234
235     print(len(predbox))
236
237     for i in range(len(predbox)):
238         xmin = int(predbox[i].xmin)
239         ymin = int(predbox[i].ymin)
240         xmax = int(predbox[i].xmax)
241         ymax = int(predbox[i].ymax)
242         classId = predbox[i].classId
243         score = predbox[i].score
244         head = predbox[i].head
245
246         cv2.rectangle(orig_img, (xmin, ymin), (xmax, ymax), (0, 255, 0), 2)
247         ptext = (xmin, ymin)
248         title = CLASSES[classId] + ":%d:%.2f" % (head, score)
249         cv2.putText(orig_img, title, ptext, cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,
0, 255), 2, cv2.LINE_AA)
250
251     cv2.imwrite('./test_rknn_result.jpg', orig_img)
252     # cv2.imshow("test", origimg)
253     # cv2.waitKey(0)
254

```

1.3 在开发板上进行测试

代码块

```

1  ./build/yolov8_img ./weight/yolov8s.float.rknn ./bus.jpg
2  ./build/yolov8_img ./weight/yolov8s.int.rknn ./bus.jpg

```

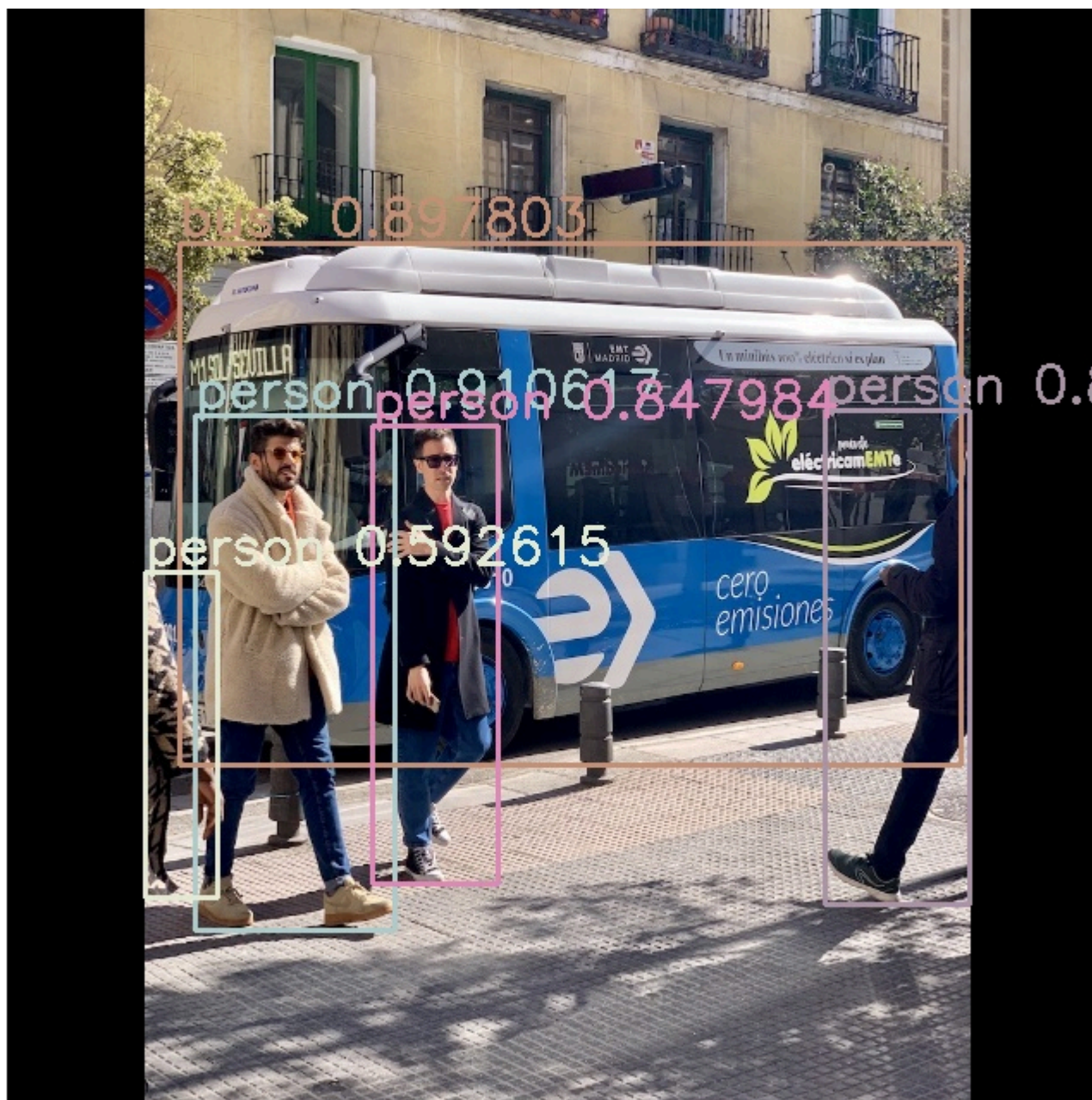
查看结果

代码块

```

1  ./build/yolov8_img weights/yolov8s.float.rknn images/bus.jpg
2  ./build/yolov8_img weights/yolov8s.int.rknn images/bus.jpg

```



2. 加速优化测试

2.0 letterbox



Letterbox 的核心作用是：保持图像的原始宽高比，防止物体变形。

- 它的逻辑：计算原图与目标尺寸的比例，选择较小的比例进行等比缩放。缩放后，图像可能无法完全填满目标尺寸，于是用灰色（通常是 `114,114,114`）或黑色（`0,0,0`）在四周填充黑边，使图像居中。
- 为什么必须用它？如果直接将一张 `4032x3024` 的照片强制拉伸到 `640x640`，正方形可能会变成椭圆，人脸会被拉宽或压扁。模型在训练时如果没见过这种形变的特征，推理时就会漏检或误检。

2.1 模型量化

浮点模型测试

代码块

```
1 ./build/yolov8_video ./weights/yolov8s.float.rknn ./place.mp4
2
3 # 监控NPU
4 watch -n 1 "cat /sys/kernel/debug/rknpu/load"
5 # 监控CPU
6 htop
```

```
[NN_INFO] Method1 Time: 2.544000ms, 133.315000ms, 135.860000ms
[NN_INFO] Method1 FPS: 393.081761, 7.501031, 7.360518
[NN_INFO] Method2 Time:1080.354004ms, FPS:7.404980, Frame Count:8
```

```
Every 1.0s: cat /sys/kernel/debug/rknpu/load
```

```
NPU load: Core0: 56%, Core1: 0%, Core2: 0%,
```

量化模型测试

代码块

```
1 ./build/yolov8_video ./weights/yolov8s.int.rknn ./place.mp4
2
3 # 监控NPU
4 watch -n 1 "cat /sys/kernel/debug/rknpu/load"
5 # 监控CPU
6 htop
```

```
[NN_INFO] Method1 Time: 1.693000ms, 56.641000ms, 58.335000ms
[NN_INFO] Method1 FPS: 590.667454, 17.655056, 17.142367
[NN_INFO] Method2 Time:1010.700989ms, FPS:16.820009, Frame Count:17
[NN_INFO] Method2 Time:1.693000ms, 56.641000ms, 58.335000ms
```

```
Every 1.0s: cat /sys/kernel/debug/rknpu/load
```

```
NPU load: Core0: 57%, Core1: 0%, Core2: 0%,
```

2.2 RGA加速

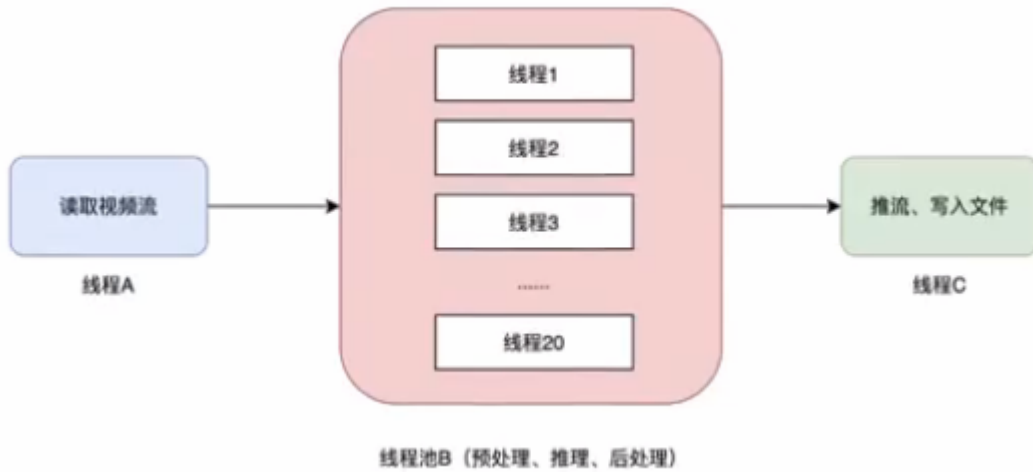
代码块

```
1 ./build/yolov8_video ./weights/yolov8s.float.rknn ./place.mp4
2
3 # 监控NRGA
4 watch -n 1 "cat /sys/kernel/debug/rkrga/load"
5 # 都监控
6 watch -n 1 "cat /sys/kernel/debug/rknpu/load; echo '---'; cat
  /sys/kernel/debug/rkrga/load"
7 # 监控CPU
8 htop
```

2.3 线程池加速

因为RK3588包含3个NPU核心，支持三个核心合作、双核合作以及独立工作。我们可以使用线程池来加速：

线程池是一种常见的并发编程模型，它用于管理和复用线程，以提高多线程程序的性能和效率。线程通常包含以下主要组件：



1.任务队列(TaskQueue): 任务队列用于存储待执行的任务。这些任务可以是函数、方法或其他需要在多线程环境中执行的操作。

2.线程池管理器: 线程池管理器负责创建、销毁和管理线程。它会监视任务队列中的任务，根据需要创建线程，执行任务，并在任务完成后将线程放回池中以供重用。

3.工作线程(Worker Threads): 工作线程是线程池中的实际执行单元。它们会从任务队列中获取任务，并执行这些任务。一个线程池通常包含多个工作线程。

在使用线程池时，需要考虑线程数、任务队列大小和任务类型等参数，以确保线程池的性能和稳定性。

浮点模型测试

代码块

```
1  ./build/yolov8_thread_pool ./weights/yolov8s.float.rknn ./place.mp4
   1/5/10/12/15/18
2
3  # 监控NPU
4  watch -n 1 "cat /sys/kernel/debug/rknpu/load"
5  # 监控CPU
6  htop
```

```

root@topeet:~/yolov8# ./build/yolov8_thread_pool ./weights/yolov8s.float.rknn videos/palace.mp4 1
[NN_INFO] rknn_init success!
[NN_INFO] RKNN API version: 1.5.3b6 (181ec8d8b@2023-09-12T17:11:43)
[NN_INFO] RKNN Driver version: 0.9.8
[NN_INFO] model input num: 1, output num: 6
[NN_INFO] input tensors:
[NN_INFO]   index=0, name=data, n_dims=4, dims=[1, 640, 640, 3], n_elems=1228800, size=2457600, fmt=NHWC, type=FP16, scale=1.000000
[NN_INFO] output tensors:
[NN_INFO]   index=0, name=reg1, n_dims=4, dims=[1, 1, 4, 6400], n_elems=25600, size=51200, fmt=NCHW, type=FP16, scale=1.000000
[NN_INFO]   index=1, name=cls1, n_dims=4, dims=[1, 80, 80, 80], n_elems=512000, size=1024000, fmt=NCHW, type=FP16, scale=1.000000
[NN_INFO]   index=2, name=reg2, n_dims=4, dims=[1, 1, 4, 1600], n_elems=6400, size=12800, fmt=NCHW, type=FP16, scale=1.000000
[NN_INFO]   index=3, name=cls2, n_dims=4, dims=[1, 80, 40, 40], n_elems=128000, size=256000, fmt=NCHW, type=FP16, scale=1.000000
[NN_INFO]   index=4, name=reg3, n_dims=4, dims=[1, 1, 4, 400], n_elems=1600, size=3200, fmt=NCHW, type=FP16, scale=1.000000
[NN_INFO]   index=5, name=cls3, n_dims=4, dims=[1, 80, 20, 20], n_elems=32000, size=64000, fmt=NCHW, type=FP16, scale=1.000000
[NN_WARNING] yolov8 output tensor type is float16, want type set to float32
[NN_INFO] Video size: 1280 x 720, fps: 30
=== yolov8 Meshgrid Generate success!
[NN_INFO] Method2 Time:1104.895996ms, FPS:7.240501, Frame Count:8
[NN_INFO] Method2 Time:1075.817993ms, FPS:7.436203, Frame Count:8
[NN_INFO] Method2 Time:1061.312988ms, FPS:7.537833, Frame Count:8

```

```

every 1.0s: cat /sys/kernel/debug/rknpu/load; echo
NPU load:  Core0: 57%, Core1:  0%, Core2:  0%,
--
num of scheduler = 3

```

```

scale=1.000000
[NN_INFO] output tensors:
[NN_INFO]   index=0, name=reg1, n_dims=4, dims=[1, 1, 4, 6400], n_elems=25600, size=51200, fmt=NCHW, type=FP16, scale=1.000000
[NN_INFO]   index=1, name=cls1, n_dims=4, dims=[1, 80, 80, 80], n_elems=512000, size=1024000, fmt=NCHW, type=FP16, scale=1.000000
[NN_INFO]   index=2, name=reg2, n_dims=4, dims=[1, 1, 4, 1600], n_elems=6400, size=12800, fmt=NCHW, type=FP16, scale=1.000000
[NN_INFO]   index=3, name=cls2, n_dims=4, dims=[1, 80, 40, 40], n_elems=128000, size=256000, fmt=NCHW, type=FP16, scale=1.000000
[NN_INFO]   index=4, name=reg3, n_dims=4, dims=[1, 1, 4, 400], n_elems=1600, size=3200, fmt=NCHW, type=FP16, scale=1.000000
[NN_INFO]   index=5, name=cls3, n_dims=4, dims=[1, 80, 20, 20], n_elems=32000, size=64000, fmt=NCHW, type=FP16, scale=1.000000
[NN_WARNING] yolov8 output tensor type is float16, want type set to float32
[NN_INFO] Video size: 1280 x 720, fps: 30
=== yolov8 Meshgrid Generate success!
[NN_INFO] Method2 Time:1086.430054ms, FPS:15.647578, Frame Count:17
[NN_INFO] Method2 Time:1077.880005ms, FPS:26.904665, Frame Count:29
[NN_INFO] Method2 Time:1001.094971ms, FPS:32.963905, Frame Count:33
[NN_INFO] Method2 Time:1017.937012ms, FPS:31.436129, Frame Count:32
[NN_INFO] Method2 Time:1000.112976ms, FPS:28.996723, Frame Count:29
[NN_INFO] Method2 Time:1125.401978ms, FPS:29.322857, Frame Count:33
[NN_INFO] Method2 Time:1039.704956ms, FPS:36.548832, Frame Count:38
[NN_INFO] Method2 Time:1097.081055ms, FPS:29.168310, Frame Count:32
[NN_INFO] Method2 Time:1049.892944ms, FPS:34.289211, Frame Count:36
[NN_INFO] Video end.
[NN_INFO] Method2 Time:1020.540011ms, FPS:31.255671, Frame Count:33

```

```

Every 1.0s: cat /sys/kernel/debug/rknpu/load; echo '---'; cat /
NPU load:  Core0: 95%, Core1: 92%, Core2: 92%,
---
num of scheduler = 3
===== load =====
scheduler[0]: rga3_core0
            load = 0%
-----
scheduler[1]: rga3_core1
            load = 0%
-----
scheduler[2]: rga2
            load = 0%
-----

```

量化模型测试

代码块

```

1  ./build/yolov8_thread_pool ./weights/yolov8s.int.rknn ./place.mp4
   1/5/10/12/15/18
2
3  # 监控NPU
4  watch -n 1 "cat /sys/kernel/debug/rknpu/load"
5  # 监控CPU
6  htop

```

```

root@topeet:~/yolov8# ./build/yolov8_thread_pool ./weights/yolov8s.int.rknn videos/palace.mp4 1
[NN_INFO] rknn_init success!
[NN_INFO] RKNN API version: 1.5.3b6 (181ec8d8b@2023-09-12T17:11:43)
[NN_INFO] RKNN Driver version: 0.9.8
[NN_INFO] model input num: 1, output num: 6
[NN_INFO] input tensors:
[NN_INFO]   index=0, name=data, n_dims=4, dims=[1, 640, 640, 3], n_elems=1228800, size=1228800, fmt=NHWC, t
, scale=0.003922
[NN_INFO] output tensors:
[NN_INFO]   index=0, name=reg1, n_dims=4, dims=[1, 1, 4, 6400], n_elems=25600, size=25600, fmt=NCHW, type=I
le=0.045245
[NN_INFO]   index=1, name=cls1, n_dims=4, dims=[1, 80, 80, 80], n_elems=512000, size=512000, fmt=NCHW, type
ale=0.086021
[NN_INFO]   index=2, name=reg2, n_dims=4, dims=[1, 1, 4, 1600], n_elems=6400, size=6400, fmt=NCHW, type=INT
=0.049624
[NN_INFO]   index=3, name=cls2, n_dims=4, dims=[1, 80, 40, 40], n_elems=128000, size=128000, fmt=NCHW, type
ale=0.098615
[NN_INFO]   index=4, name=reg3, n_dims=4, dims=[1, 1, 4, 400], n_elems=1600, size=1600, fmt=NCHW, type=INT8
0.048530
[NN_INFO]   index=5, name=cls3, n_dims=4, dims=[1, 80, 20, 20], n_elems=32000, size=32000, fmt=NCHW, type=I
e=0.150579
[NN_INFO] Video size: 1280 x 720, fps: 30
=== yolov8 Meshgrid Generate success!
[NN_INFO] Method2 Time:1030.104004ms, FPS:14.561636, Frame Count:15
[NN_INFO] Method2 Time:1036.562988ms, FPS:16.400352, Frame Count:17
[NN_INFO] Method2 Time:1041.699951ms, FPS:16.319477, Frame Count:17
[NN_INFO] Method2 Time:1041.536011ms, FPS:16.322048, Frame Count:17
[NN_INFO] Method2 Time:1032.551025ms, FPS:16.464077, Frame Count:17
[NN_INFO] Method2 Time:1037.326050ms, FPS:16.388288, Frame Count:17
[NN_INFO] Method2 Time:1038.610962ms, FPS:16.368015, Frame Count:17

```

```
Every 1.0s: cat /sys/kernel/debug/rknpu/load; echo '---'; cat
NPU load: Core0: 55%, Core1: 0%, Core2: 0%,
---
num of scheduler = 3
===== load =====
scheduler[0]: rga3_core0
            load = 0%
-----
scheduler[1]: rga3_core1
            load = 0%
```

```
[NN_INFO] Video size: 1280 x 720, fps: 24
=== yolov8 Meshgrid Generate success!
[NN_INFO] Method2 Time:1053.176025ms, FPS:51.273479, Frame Count:54
[NN_INFO] Method2 Time:1052.640015ms, FPS:76.949387, Frame Count:81
[NN_INFO] Method2 Time:1127.631958ms, FPS:29.264870, Frame Count:33
[NN_INFO] Method2 Time:1026.187012ms, FPS:113.039818, Frame Count:116
[NN_INFO] Method2 Time:1034.229004ms, FPS:79.286110, Frame Count:82
[NN_INFO] Method2 Time:1008.864014ms, FPS:78.305893, Frame Count:79
[NN_INFO] Method2 Time:1011.710022ms, FPS:74.131912, Frame Count:75
[NN_INFO] Method2 Time:1007.331970ms, FPS:66.512337, Frame Count:67
[NN_INFO] Method2 Time:1001.565979ms, FPS:84.867104, Frame Count:85
[NN_INFO] Method2 Time:1021.508972ms, FPS:71.462906, Frame Count:73
[NN_INFO] Method2 Time:1003.841003ms, FPS:79.693893, Frame Count:80
[NN_INFO] Method2 Time:1031.698975ms, FPS:72.695625, Frame Count:75
[NN_INFO] Method2 Time:1019.661987ms, FPS:72.573067, Frame Count:74
```

```
Every 1.0s: cat /sys/kernel/debug/rknpu/load; echo '---'; cat
NPU load: Core0: 95%, Core1: 94%, Core2: 92%,
---
num of scheduler = 3
===== load =====
scheduler[0]: rga3_core0
            load = 0%
-----
scheduler[1]: rga3_core1
            load = 0%
-----
scheduler[2]: rga2
            load = 0%
-----
```

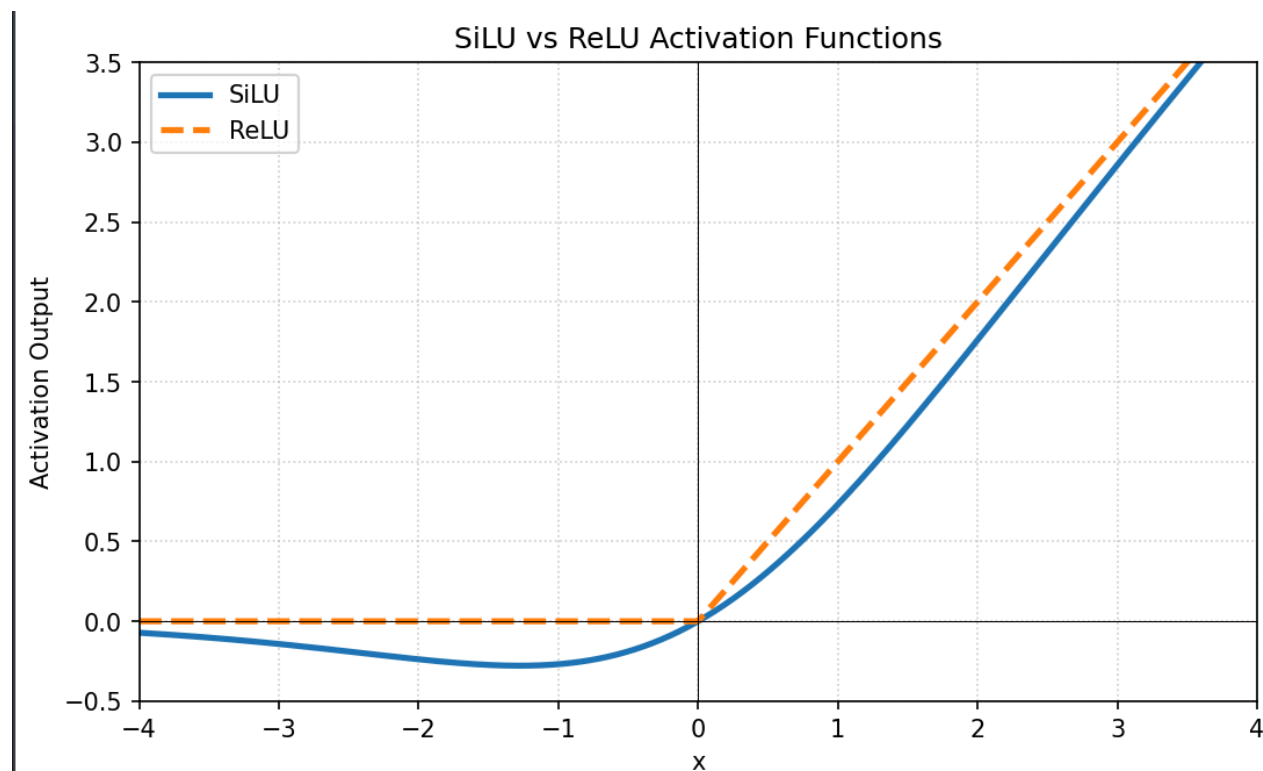
2.4 更换激活函数

原版的yolov8使用的激活函数的SiLU,我们替换成更高效的ReLU.

重点：替换激活函数以后，原本的pytorch模型需要重新训练，然后再导出onnx，然后再转换成rknn模型。

$$\text{ReLU}(x) = \max(0, x)$$

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$



具体修改：

ultralytics\nn\modules\conv.py，把SiLU改成ReLU就行。

```
22
23 class Conv(nn.Module):
24     """Standard convolution with args(ch_in, ch_out, kernel, stride, padding)
25     # default_act = nn.SiLU() # default activation
26     default_act = nn.ReLU() # default activation
27
28     def __init__(self, c1, c2, k=1, s=1, p=None, g=1, d=1, act=True):
29         """Initialize Conv layer with given arguments including activation.
30         super().__init__()
```

注意：然后需要重新训练模型，训练时不需要指定预训练模型，需要重头训练！

创建tran.py文件重头训练yolov8模型

代码块

```
1 from ultralytics import YOLO
```

```

2
3 # 1. 加载模型
4 model = YOLO("yolov8s.yaml")
5
6 if __name__ == '__main__':
7     # 2. 开始训练
8     results = model.train(
9         data="person_data.yaml",           # 数据集配置文件路径
10        epochs=300,                         # 建议将训练轮数提升至300, 配合早停机制更合理
11        imgsiz=640,                        # 明确指定输入图像尺寸为640x640
12
13        batch=-1,                          # 设置为-1开启AutoBatch, 自动探测3060Ti能跑
14        workers=4,                         # 设为4以充分利用CPU多线程, 避免GPU等待数据
15
16        patience=50,                       # 如果验证集指标连续50轮不提升则提前停止, 防止
17        device=0,                          # 明确指定使用第一张显卡进行训练
18
19        task="detect",                     # 任务类型: 目标检测
20        mode="train",                      # 模式: 训练
21        verbose=True                       # 显示详细训练日志
22    )
23
24    print("训练完成! ")
25

```

person_data.yaml

代码块

```

1 path: E:\YOLOV8\ultralytics\datasets
2 train: images/train
3 val: images/val
4
5 # Classes, 类别: 0:pedestrians, 1:riders, 2:partially-visible persons, 3:ignore
  regions, 4:crowd
6 names:
7     0: pedestrian
8     1: riders
9     2: partially-visible-persons
10    3: ignore-regions
11    4: crowd
12

```

然后再次导出onnx，导出rknn模型，然后来测试。

注意：

转换onnx模型的时候，需要把SiLU修改为ReLU

转换rknn模型的时候，需要修改CLASSES类别

代码块

```
1 CLASSES = ['pedestrians', 'riders', 'partially-visible-person', 'ignore-  
regions', 'crowd']
```

板端部署的代码也需要进行修改

代码块

```
1 // postprocess.cpp  
2 static int class_num = 5;  
3 // yolov8_custom.cpp  
4 static std::vector<std::string> g_classes = {  
5 "pedestrians", "riders", "partially-visible-person", "ignore-regions",  
"crowd"};
```

然后再次进行测试，查看性能。

量化模型测试

代码块

```
1 ./build/yolov8_thread_pool weights/person_s_relu.int.rknn videos/bj_full.mp4  
15  
2  
3 # 监控NPU  
4 watch -n 1 "cat /sys/kernel/debug/rknpu/load"  
5 # 监控CPU  
6 htop
```

s模型

```
[NN_INFO] Method2 Time:1035.868042ms, FPS:88.814400, Frame Count:92
[NN_INFO] Method2 Time:1016.302979ms, FPS:91.508148, Frame Count:93
[NN_INFO] Method2 Time:1082.970947ms, FPS:100.649048, Frame Count:109
[NN_INFO] Method2 Time:1042.745972ms, FPS:95.900635, Frame Count:100
[NN_INFO] Method2 Time:1055.713013ms, FPS:80.514305, Frame Count:85
[NN_INFO] Method2 Time:1065.737061ms, FPS:101.338326, Frame Count:108
[NN_INFO] Method2 Time:1029.224976ms, FPS:96.188881, Frame Count:99
[NN_INFO] Method2 Time:1007.265015ms, FPS:88.358078, Frame Count:89
[NN_INFO] Method2 Time:1052.287964ms, FPS:91.229782, Frame Count:96
[NN_INFO] Method2 Time:1012.237000ms, FPS:92.863632, Frame Count:94
[NN_INFO] Method2 Time:1010.075012ms, FPS:99.002548, Frame Count:100
[NN_INFO] Method2 Time:1017.432983ms, FPS:90.423653, Frame Count:92
[NN_INFO] Method2 Time:1016.489014ms, FPS:86.572502, Frame Count:88
[NN_INFO] Method2 Time:1059.661011ms, FPS:89.651314, Frame Count:95
[NN_INFO] Method2 Time:1006.309021ms, FPS:94.404396, Frame Count:95
[NN_INFO] Method2 Time:1106.970947ms, FPS:87.626511, Frame Count:97
[NN_INFO] Method2 Time:1040.069946ms, FPS:93.262962, Frame Count:97
[NN_INFO] Method2 Time:1010.091980ms, FPS:93.060829, Frame Count:94
```

```
Every 1.0s: cat /sys/kernel/debug/rknpu/load; echo '---'; cat /sys/k
NPU load: Core0: 94%, Core1: 92%, Core2: 91%,
---
num of scheduler = 3
===== load =====
scheduler[0]: rga3_core0
          load = 0%
-----
scheduler[1]: rga3_core1
          load = 0%
-----
scheduler[2]: rga2
          load = 0%
-----
```

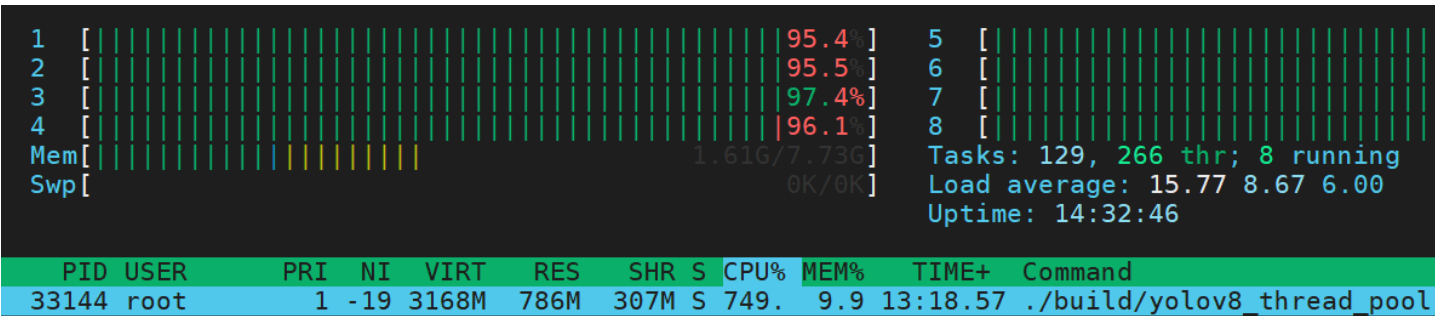
n模型

```
[NN_INFO] Method2 Time:1001.729980ms, FPS:102.822121, Frame Count:103
[NN_INFO] Method2 Time:1030.637939ms, FPS:124.194916, Frame Count:128
[NN_INFO] Method2 Time:1006.208008ms, FPS:130.191757, Frame Count:131
[NN_INFO] Method2 Time:1029.541992ms, FPS:115.585381, Frame Count:119
[NN_INFO] Method2 Time:1002.531982ms, FPS:121.691879, Frame Count:122
[NN_INFO] Method2 Time:1026.267944ms, FPS:119.851746, Frame Count:123
[NN_INFO] Method2 Time:1055.977051ms, FPS:121.214752, Frame Count:128
[NN_INFO] Method2 Time:1030.244995ms, FPS:126.183586, Frame Count:130
[NN_INFO] Method2 Time:1017.750000ms, FPS:122.819946, Frame Count:125
[NN_INFO] Method2 Time:1010.088013ms, FPS:123.751595, Frame Count:125
[NN_INFO] Method2 Time:1025.500977ms, FPS:121.891640, Frame Count:125
[NN_INFO] Method2 Time:1006.187988ms, FPS:121.249702, Frame Count:122
[NN_INFO] Method2 Time:1030.282959ms, FPS:119.384674, Frame Count:123
[NN_INFO] Method2 Time:1048.625000ms, FPS:125.879128, Frame Count:132
[NN_INFO] Method2 Time:1002.455017ms, FPS:133.671829, Frame Count:134
[NN_INFO] Method2 Time:1046.239990ms, FPS:113.740639, Frame Count:119
[NN_INFO] Method2 Time:1021.731018ms, FPS:119.405205, Frame Count:122
[NN_INFO] Method2 Time:1034.031982ms, FPS:126.688538, Frame Count:131
[NN_INFO] Method2 Time:1035.015015ms, FPS:123.669708, Frame Count:128
[NN_INFO] Method2 Time:1000.494995ms, FPS:125.937668, Frame Count:126
[NN_INFO] Method2 Time:1000.843994ms, FPS:128.891220, Frame Count:129
[NN_INFO] Method2 Time:1005.007996ms, FPS:118.407021, Frame Count:119
[NN_INFO] Method2 Time:1037.173950ms, FPS:121.483955, Frame Count:126
```

```

Every 1.0s: cat /sys/kernel/debug/rknpu/load; echo '---'; cat /sys/ke
NPU load:  Core0: 67%, Core1: 51%, Core2: 39%,
---
num of scheduler = 3
===== load =====
scheduler[0]: rga3_core0
            load = 0%
-----
scheduler[1]: rga3_core1
            load = 0%
-----
scheduler[2]: rga2
            load = 0%
-----

```



还有空间，但是CPU已经满了，就没办法了